

JAVA GRUNDLAGEN

VARIABLEN

! ☕

ZAP!

DREH!

Variablen speichern Daten!

int

String

boolean

alter = 25

name = "Leo"

isReady = true

name

"Leo"

words

KONTROLLSTRUKTUREN

IF-ELSE

condition

WAHR (true)

FALSCH (false)

Codeblock 1

```
if (a > b) {  
  console.log...  
}
```

Codeblock 2

```
else {  
  alert...  
}
```

LOOPS (Schleifen)

Java

If/Else & Schleifen lenken den Fluss!

public class malic main {
 ...
}

KLICK!

for(i=0; i<10; i++) { ... }

Aktion 1 Aktion 2 Aktion 3 ... Aktion 10

Aktion 8 Aktion 9

FOR LOOP

for(i=0; i<10; i++)

die Programmiersprache Java

Java ist eine Programmiersprache, mit der man zum Beispiel **Anwendungen, Webserver, Spiele und große Unternehmensprogramme** entwickeln kann.

die Programmiersprache Java

wichtige Eigenschaften

Java ist objektorientiert.

Programme werden in sogenannte **Klassen** und **Objekte** aufgeteilt. Ein Objekt kann Eigenschaften und Funktionen besitzen.

Beispiel: Ein Objekt **Auto** könnte die Eigenschaft **farbe** und die Funktion **fahren()** haben.

die Programmiersprache Java

wichtige Eigenschaften

Java ist weitgehend plattformunabhängig.

Java-Programme werden nicht direkt für Windows, macOS oder Linux übersetzt. Sie laufen zunächst auf der **Java Virtual Machine**, kurz **JVM**. Dadurch kann derselbe Java-Code auf verschiedenen Betriebssystemen ausgeführt werden.

die Programmiersprache Java

wichtige Eigenschaften

Java ist statisch typisiert.

Bei jeder Variablen wird angegeben, welche Art von Wert sie speichern soll:

JAVA

```
int alter = 20;  
String name = "Mina";
```

`int` steht für eine ganze Zahl, `String` für Text. Dadurch kann Java viele Fehler bereits vor dem Start des Programms erkennen.

die Programmiersprache Java

wichtige Eigenschaften

Java besitzt eine feste und klare Syntax.

Anweisungen enden meistens mit einem Semikolon. Blöcke werden durch geschweifte Klammern gekennzeichnet:

JAVA

```
if (alter >= 18) {  
    System.out.println("Volljährig");  
}
```

die Programmiersprache Java

wichtige Eigenschaften

Java ist nicht dasselbe wie JavaScript.

Obwohl die Namen ähnlich klingen, handelt es sich um zwei verschiedene Programmiersprachen mit unterschiedlichen Einsatzbereichen.

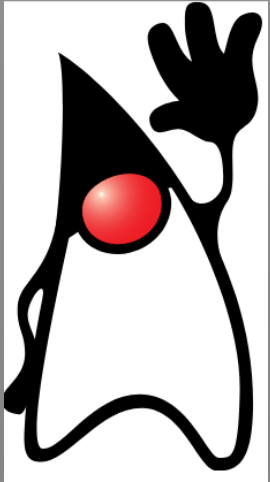
die Programmiersprache Java

Entstehungsgeschichte

- **1991:** Bei Sun Microsystems begann ein Team um **James Gosling** im sogenannten *Green Project* mit der Entwicklung einer neuen Programmiersprache. Sie hieß zunächst **Oak**.
- **1995:** Die Sprache wurde in **Java** umbenannt und offiziell vorgestellt. Das Ziel war, Programme unabhängig vom jeweiligen Computer und Betriebssystem ausführen zu können.
- **1996:** Sun veröffentlichte die erste Version **Java 1.0**. Das zentrale Prinzip lautete: „**Write once, run anywhere**“ – ein Programm sollte einmal geschrieben und anschließend auf verschiedenen Systemen ausgeführt werden können.
- **Ende der 1990er-Jahre:** Java wurde durch sogenannte **Applets** im Web bekannt. Außerdem setzte sich Java zunehmend bei Unternehmenssoftware und auf Servern durch.
- **2006:** Sun stellte große Teile der Java-Technologie als **Open Source** zur Verfügung. Daraus entstand unter anderem das Projekt **OpenJDK**.
- **2010:** Das Unternehmen **Oracle** übernahm Sun Microsystems und damit auch die Weiterentwicklung von Java.
- **Seit 2017:** Neue Java-Versionen erscheinen regelmäßig, ungefähr alle sechs Monate. Einige Versionen erhalten als **Long-Term-Support-Versionen (LTS)** besonders lange Unterstützung.

die Programmiersprache Java

Entstehungsgeschichte



Duke



James Gosling

- **1991:** Bei Sun Microsystems begann ein Team um **James Gosling** im sogenannten *Green Project* mit der Entwicklung einer neuen Programmiersprache. Sie hieß zunächst **Oak**.
- **1995:** Die Sprache wurde in **Java** umbenannt und offiziell vorgestellt. Das Ziel war, Programme unabhängig vom jeweiligen Computer und Betriebssystem ausführen zu können.
- **1996:** Sun veröffentlichte die erste Version **Java 1.0**. Das zentrale Prinzip lautete: „**Write once, run anywhere**“ – ein Programm sollte einmal geschrieben und anschließend auf verschiedenen Systemen ausgeführt werden können.
- **Ende der 1990er-Jahre:** Java wurde durch sogenannte **Applets** im Web bekannt. Außerdem setzte sich Java zunehmend bei Unternehmenssoftware und auf Servern durch.
- **2006:** Sun stellte große Teile der Java-Technologie als **Open Source** zur Verfügung. Daraus entstand unter anderem das Projekt **OpenJDK**.
- **2010:** Das Unternehmen **Oracle** übernahm Sun Microsystems und damit auch die Weiterentwicklung von Java.
- **Seit 2017:** Neue Java-Versionen erscheinen regelmäßig, ungefähr alle sechs Monate. Einige Versionen erhalten als **Long-Term-Support-Versionen (LTS)** besonders lange Unterstützung.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Eine Programmiersprache besteht aus verschiedenen Regeln und Elementen. Mit ihnen können Probleme beschrieben und Lösungen so formuliert werden, dass ein Computer sie ausführen kann.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Syntax

Die Syntax legt fest, *wie* Programmcode geschrieben werden muss. Sie beschreibt zum Beispiel, wo Klammern, Semikolons oder Anführungszeichen stehen müssen.

JAVA

```
int alter = 18;
```

Ein fehlendes Semikolon kann zu einem Syntaxfehler führen.

Gesamtheit der Regeln, die innerhalb einer Programmiersprache zur exakten Formulierung eines Programms (4) erforderlich sind

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Semantik

Die Semantik beschreibt, *was* ein Befehl bedeutet und welche Wirkung er hat. Der Befehl

JAVA

```
alter = alter + 1;
```

erhöht den Wert der Variablen `alter` um 1.

Bedeutung, Inhalt (eines Wortes, Satzes oder Textes)

<https://www.duden.de/rechtschreibung/Semantik>

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Variablen und Konstanten

Variablen speichern Werte, die sich während der Programmausführung ändern können.

JAVA

```
int punkte = 10;  
punkte = 15;
```

Eine Konstante besitzt dagegen einen festgelegten Wert, der nicht verändert werden soll.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Datentypen

Datentypen legen fest, welche Art von Wert gespeichert wird. Häufige Datentypen sind:

- `int` für ganze Zahlen
- `double` für Kommazahlen
- `boolean` für `true` oder `false`
- `char` für ein einzelnes Zeichen
- `String` für Text

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Operatoren und Ausdrücke

Operatoren führen Berechnungen oder Vergleiche durch.

JAVA

```
a + b;      // Addition  
a > b;      // Vergleich  
a == b;     // Prüfung auf Gleichheit
```

Ein Ausdruck kann aus Werten, Variablen und Operatoren bestehen.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Anweisungen

Eine Anweisung beschreibt eine einzelne Aktion, zum Beispiel eine Berechnung oder eine Ausgabe.

JAVA

```
System.out.println("Hallo");
```

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Kontrollstrukturen

Kontrollstrukturen bestimmen, in welcher Reihenfolge Anweisungen ausgeführt werden.

- **Sequenz:** Anweisungen werden nacheinander ausgeführt.
- **Auswahl:** Mit `if` oder `switch` wird eine Entscheidung getroffen.
- **Wiederholung:** Mit Schleifen wie `for` oder `while` werden Anweisungen mehrfach ausgeführt.

JAVA

```
if (alter >= 18) {  
    System.out.println("Volljährig");  
}
```

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Funktionen oder Methoden

Funktionen und Methoden fassen mehrere Anweisungen zu einer wiederverwendbaren Einheit zusammen. Sie können Eingabewerte erhalten und ein Ergebnis zurückgeben.

JAVA

```
static int addieren(int a, int b) {  
    return a + b;  
}
```

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Datenstrukturen

Datenstrukturen helfen dabei, mehrere Werte zu speichern und zu verwalten. Beispiele sind:

- Arrays
- Listen
- Mengen
- Tabellen
- Objekte

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Ein- und Ausgabe

Programme müssen häufig Daten einlesen und Ergebnisse ausgeben. Eingaben können beispielsweise von der Tastatur oder aus einer Datei kommen. Ausgaben können auf dem Bildschirm oder in einer Datei erscheinen.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Fehlerbehandlung

Programme müssen auf Fehler reagieren können, etwa auf eine ungültige Eingabe oder eine nicht gefundene Datei. Viele Programmiersprachen verwenden dafür Ausnahmen, in Java beispielsweise `try` und `catch`.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Kommentare

Kommentare erklären den Quellcode und werden vom Computer nicht ausgeführt.

JAVA

```
// Das Alter des Benutzers  
int alter = 20;
```

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Klassen und Objekte

In objektorientierten Programmiersprachen wie Java werden Daten und zugehörige Funktionen häufig in Klassen zusammengefasst. Aus einer Klasse können konkrete Objekte erzeugt werden.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Bibliotheken und Module

Bibliotheken enthalten fertige Funktionen und Klassen. Dadurch müssen häufig benötigte Lösungen, zum Beispiel für Dateien oder Netzwerke, nicht vollständig selbst programmiert werden.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Die wichtigsten Bestandteile einer Programmiersprache sind:

- Regeln für die Schreibweise (**Syntax**)
- Bedeutung der Befehle (**Semantik**)
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Anweisungen und Kontrollstrukturen
- Funktionen oder Methoden
- Datenstrukturen
- Ein- und Ausgabe
- Fehlerbehandlung
- Kommentare und Bibliotheken

Eine Programmiersprache stellt also Regeln und Werkzeuge bereit, mit denen ein Algorithmus eindeutig in ausführbaren Programmcode umgesetzt werden kann.

die Programmiersprache Java

wichtige Bestandteile einer Programmiersprache

Die wichtigsten Bestandteile einer Programmiersprache sind:

- Regeln für die Schreibweise (**Syntax**)
- Bedeutung der Befehle (**Semantik**)
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Anweisungen und Kontrollstrukturen
- Funktionen oder Methoden
- Datenstrukturen
- Ein- und Ausgabe
- Fehlerbehandlung
- Kommentare und Bibliotheken

Eine Programmiersprache stellt also Regeln und Werkzeuge bereit, mit denen ein Algorithmus eindeutig in ausführbaren Programmcode umgesetzt werden kann.

die Programmiersprache Java

Vergleichsoperatoren

Vergleichsoperatoren vergleichen zwei Werte miteinander. Das Ergebnis ist immer ein Wahrheitswert vom Datentyp `boolean`:

- `true` – die Bedingung ist erfüllt
- `false` – die Bedingung ist nicht erfüllt

Sie werden häufig in `if`-Anweisungen und Schleifen verwendet.

die Programmiersprache Java

Vergleichsoperatoren

Die wichtigsten Vergleichsoperatoren

Operator	Bedeutung	Beispiel	Ergebnis
<code>==</code>	ist gleich	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	ist nicht gleich	<code>5 != 3</code>	<code>true</code>
<code>></code>	ist größer als	<code>8 > 3</code>	<code>true</code>
<code><</code>	ist kleiner als	<code>2 < 7</code>	<code>true</code>
<code>>=</code>	ist größer oder gleich	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	ist kleiner oder gleich	<code>4 <= 6</code>	<code>true</code>

die Programmiersprache Java

Vergleichsoperatoren

Verwendung mit Zahlen

JAVA

```
int alter = 20;  
  
if (alter >= 18) {  
    System.out.println("Volljährig");  
}
```

Die Bedingung `alter >= 18` wird geprüft. Da `20` größer als oder gleich `18` ist, wird die Ausgabe ausgeführt.

die Programmiersprache Java

Vergleichsoperatoren

Wichtiger Unterschied: `=` und `==`

Diese beiden Zeichenfolgen haben unterschiedliche Bedeutungen:

- `=` weist einer Variablen einen Wert zu.
- `==` vergleicht zwei Werte.

JAVA

```
int zahl = 10;           // Zuweisung
if (zahl == 10) {        // Vergleich
    System.out.println("Die Zahl ist 10.");
}
```

Ein häufiger Fehler ist, in einer Bedingung versehentlich `=` statt `==` zu verwenden.

die Programmiersprache Java

Vergleichsoperatoren

Kombination von Vergleichen

Vergleiche können mit logischen Operatoren verbunden werden:

JAVA

```
int alter = 25;
boolean fuehrerschein = true;

if (alter >= 18 && fuehrerschein == true) {
    System.out.println("Die Person darf fahren.");
}
```

die Programmiersprache Java

Vergleichsoperatoren

Kombination von Vergleichen

Vergleiche können mit logischen Operatoren verbunden werden:

JAVA

```
int alter = 25;
boolean fuehrerschein = true;

if (alter >= 18 && fuehrerschein == true) {
    System.out.println("Die Person darf fahren.");
}
```

Hier bedeutet:

- `&&` – und
- `||` – oder
- `!` – nicht

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Mit `if ... else if ... else` kann ein Programm mehrere Bedingungen prüfen und abhängig vom Ergebnis unterschiedliche Anweisungen ausführen.

Sie gehört zu den **Auswahlstrukturen**.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Bestandteile

JAVA

```
if (Bedingung1) {  
    // Anweisungen, wenn Bedingung1 wahr ist  
} else if (Bedingung2) {  
    // Anweisungen, wenn Bedingung2 wahr ist  
} else {  
    // Anweisungen, wenn keine Bedingung wahr ist  
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

`if`

`if` bedeutet „**wenn**“. Danach steht eine Bedingung in runden Klammern.

JAVA

```
if (alter >= 18) {  
    System.out.println("Volljährig");  
}
```

Der Programmblock wird nur ausgeführt, wenn die Bedingung den Wert `true` liefert.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

`else if`

Mit `else if` können weitere Bedingungen geprüft werden. Diese werden nur geprüft, wenn die vorherige Bedingung nicht erfüllt war.

JAVA

```
else if (alter >= 16) {  
    System.out.println("Fast volljährig");  
}
```

Es können mehrere `else if`-Blöcke verwendet werden.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

`else`

`else` bedeutet „ansonsten“. Dieser Block wird ausgeführt, wenn keine der vorherigen Bedingungen wahr ist.

JAVA

```
else {  
    System.out.println("Noch nicht 16");  
}
```

Der `else`-Block ist optional. Eine `if`-Struktur kann also auch ohne `else` verwendet werden.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Ablauf der Prüfung

Java arbeitet die Bedingungen **von oben nach unten** ab:

1. Die `if`-Bedingung wird geprüft.
2. Ist sie wahr, wird ihr Anweisungsblock ausgeführt.
3. Danach wird die gesamte Struktur verlassen.
4. Ist sie falsch, wird die nächste `else if`-Bedingung geprüft.
5. Wenn keine Bedingung wahr ist, wird der `else`-Block ausgeführt.

Wichtig ist:

Es wird höchstens **ein** Block ausgeführt.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Beispiel: Altersprüfung

JAVA

```
int alter = 17;

if (alter >= 18) {
    System.out.println("Volljährig");
} else if (alter >= 16) {
    System.out.println("Mindestens 16 Jahre alt");
} else {
    System.out.println("Unter 16 Jahre alt");
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Da `alter` den Wert `17` besitzt, ist die erste Bedingung falsch. Die zweite Bedingung ist wahr. Deshalb erscheint:

TEXT

Mindestens 16 Jahre alt

Der `else`-Block wird in diesem Fall nicht mehr ausgeführt.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Beispiel: Notenbewertung

JAVA

```
int punkte = 78;

if (punkte >= 90) {
    System.out.println("Sehr gut");
} else if (punkte >= 75) {
    System.out.println("Gut");
} else if (punkte >= 60) {
    System.out.println("Befriedigend");
} else if (punkte >= 50) {
    System.out.println("Ausreichend");
} else {
    System.out.println("Nicht bestanden");
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Bei 78 Punkten wird "Gut" ausgegeben.

Die Reihenfolge ist entscheidend. Würde zuerst `punkte >= 50` geprüft, wären auch 78 Punkte bereits in diesem Block und die genaueren Prüfungen würden nicht mehr erreicht.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Mehrere Bedingungen verbinden

Mit logischen Operatoren können mehrere Bedingungen kombiniert werden:

JAVA

```
int alter = 25;
boolean fuehrerschein = true;

if (alter >= 18 && fuehrerschein) {
    System.out.println("Die Person darf fahren.");
} else {
    System.out.println("Die Voraussetzungen sind nicht erfüllt.");
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Bei einer zusammengehörenden `if ... else if ... else`-Struktur wird höchstens ein Block ausgeführt:

JAVA

```
if (zahl > 0) {  
    System.out.println("Positiv");  
} else if (zahl < 0) {  
    System.out.println("Negativ");  
} else {  
    System.out.println("Null");  
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Bei mehreren einzelnen `if`-Anweisungen können dagegen mehrere Blöcke ausgeführt werden:

JAVA

```
if (zahl >= 0) {  
    System.out.println("Nicht negativ");  
}  
  
if (zahl <= 0) {  
    System.out.println("Nicht positiv");  
}
```

Für den Wert `0` werden hier beide Ausgaben ausgeführt.

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Fehlende runde Klammern

Falsch:

JAVA

```
if alter >= 18 {  
}
```

Richtig:

JAVA

```
if (alter >= 18) {  
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Zuweisung statt Vergleich

Falsch:

JAVA

```
if (alter = 18) {  
}
```

`=` weist einen Wert zu. Für einen Vergleich wird `==` verwendet:

JAVA

```
if (alter == 18) {  
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Semikolon direkt nach der Bedingung

Ungünstig beziehungsweise meist falsch:

JAVA

```
if (alter >= 18);  
{  
    System.out.println("Volljährig");  
}
```

Das Semikolon beendet die **if**-Anweisung sofort. Richtig ist:

JAVA

```
if (alter >= 18) {  
    System.out.println("Volljährig");  
}
```

die Programmiersprache Java

Kontrollstrukturen: Fallunterscheidungen

Zusammenfassung

Die Kontrollstruktur `if ... else if ... else` besteht aus:

- einer ersten Bedingung mit `if`,
- beliebig vielen weiteren Bedingungen mit `else if`,
- einem optionalen Standardfall mit `else`.

Sie wird verwendet, wenn abhängig von mehreren Bedingungen unterschiedliche Programmteile ausgeführt werden sollen.

| `if` prüft den ersten Fall, `else if` prüft weitere Fälle und `else` behandelt alle übrigen Fälle.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Schleifen wiederholen einen Anweisungsblock mehrfach. Wie oft die Wiederholung erfolgt, hängt von einer **Bedingung** oder einem **Zähler** ab.

Java kennt unter anderem die Schleifen:

- `for`
- `while`
- `do while`

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Die **for**-Schleife

Die **for**-Schleife eignet sich besonders, wenn bereits bekannt ist, **wie oft** ein Anweisungsblock ausgeführt werden soll.

Aufbau

JAVA

```
for (Initialisierung; Bedingung; Veränderung) {  
    // Anweisungen  
}
```

Die drei Bestandteile bedeuten:

- **Initialisierung:** Startwert wird festgelegt.
- **Bedingung:** Solange diese Bedingung wahr ist, wird die Schleife ausgeführt.
- **Veränderung:** Der Zähler wird nach jedem Durchlauf verändert.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Ausgabe:

TEXT

1
2
3
4
5

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Der Ablauf ist:

1. `i` wird mit `1` initialisiert.
2. Es wird geprüft, ob `i <= 5` gilt.
3. Der Schleifenblock wird ausgeführt.
4. `i` wird um `1` erhöht.
5. Die Bedingung wird erneut geprüft.

Die Variable `i` wird häufig als **Schleifenzähler** verwendet.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Die `while`-Schleife

Die `while`-Schleife verwendet man, wenn die Anzahl der Wiederholungen vorher nicht genau bekannt ist. Die Schleife wird ausgeführt, **solange eine Bedingung wahr ist**.

Aufbau

JAVA

```
while (Bedingung) {  
    // Anweisungen  
}
```

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
int zahl = 1;

while (zahl <= 5) {
    System.out.println(zahl);
    zahl++;
}
```

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
int zahl = 1;

while (zahl <= 5) {
    System.out.println(zahl);
    zahl++;
}
```

Auch hier werden die Zahlen von 1 bis 5 ausgegeben.

Wichtig: Die Bedingung wird **vor jedem Durchlauf** geprüft. Ist sie bereits zu Beginn falsch, wird der Schleifenblock kein einziges Mal ausgeführt.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
int zahl = 1;

while (zahl <= 5) {
    System.out.println(zahl);
    zahl++;
}
```

Auch hier werden die Zahlen von 1 bis 5 ausgegeben.

Wichtig: Die Bedingung wird **vor jedem Durchlauf** geprüft. Ist sie bereits zu Beginn falsch, wird der Schleifenblock **kein einziges Mal** ausgeführt.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Die `do while`-Schleife

Die `do while`-Schleife ähnelt der `while`-Schleife. Der entscheidende Unterschied ist:

Der Schleifenblock wird bei `do while` immer mindestens einmal ausgeführt.

Die Bedingung wird nämlich **erst nach dem Schleifenblock** geprüft.

Aufbau

JAVA

```
do {  
    // Anweisungen  
} while (Bedingung);
```

Nach der Bedingung steht bei `do while` ein Semikolon.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Die `do while`-Schleife

Die `do while`-Schleife ähnelt der `while`-Schleife. Der entscheidende Unterschied ist:

Der Schleifenblock wird bei `do while` immer mindestens einmal ausgeführt.

Die Bedingung wird nämlich **erst nach dem Schleifenblock** geprüft.

Aufbau

JAVA

```
do {  
    // Anweisungen  
} while (Bedingung);
```

Nach der Bedingung steht bei `do while` ein Semikolon.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Beispiel

JAVA

```
int zahl;  
  
do {  
    System.out.println("Zahl eingeben, 0 beendet:");  
    zahl = scanner.nextInt();  
} while (zahl != 0);
```

Die Eingabeaufforderung erscheint mindestens einmal. Danach wird geprüft, ob die Schleife wiederholt werden soll.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Vergleich der drei Schleifen

Schleife	Bedingung wird geprüft	Typische Verwendung
<code>for</code>	vor jedem Durchlauf	Anzahl der Wiederholungen ist bekannt
<code>while</code>	vor jedem Durchlauf	Wiederholung abhängig von einer Bedingung
<code>do while</code>	nach jedem Durchlauf	Schleifenblock soll mindestens einmal ausgeführt werden

die Programmiersprache Java

Kontrollstrukturen: Schleifen

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Hallo");  
}
```

```
int i = 0;
```

```
while (i < 3) {  
    System.out.println("Hallo");  
    i++;  
}
```

```
int i = 0;
```

```
do {  
    System.out.println("Hallo");  
    i++;  
} while (i < 3);
```

Alle drei Beispiele geben "Hallo" dreimal aus.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Endlosschleifen

Eine Schleife wird zur **Endlosschleife**, wenn ihre Bedingung niemals `false` wird.

JAVA

```
int i = 0;

while (i < 5) {
    System.out.println(i);
    // i++ wurde vergessen
}
```

Da `i` nie erhöht wird, bleibt die Bedingung `i < 5` immer wahr. Deshalb muss darauf geachtet werden, dass sich die Schleifenbedingung irgendwann ändert.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Mit **break** kann eine Schleife vorzeitig beendet werden:

JAVA

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

Die Ausgabe endet bei **4**.

die Programmiersprache Java

Kontrollstrukturen: Schleifen

Mit `continue` wird der aktuelle Durchlauf übersprungen. Die Schleife wird anschließend mit dem nächsten Durchlauf fortgesetzt:

JAVA

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

die Programmiersprache Java

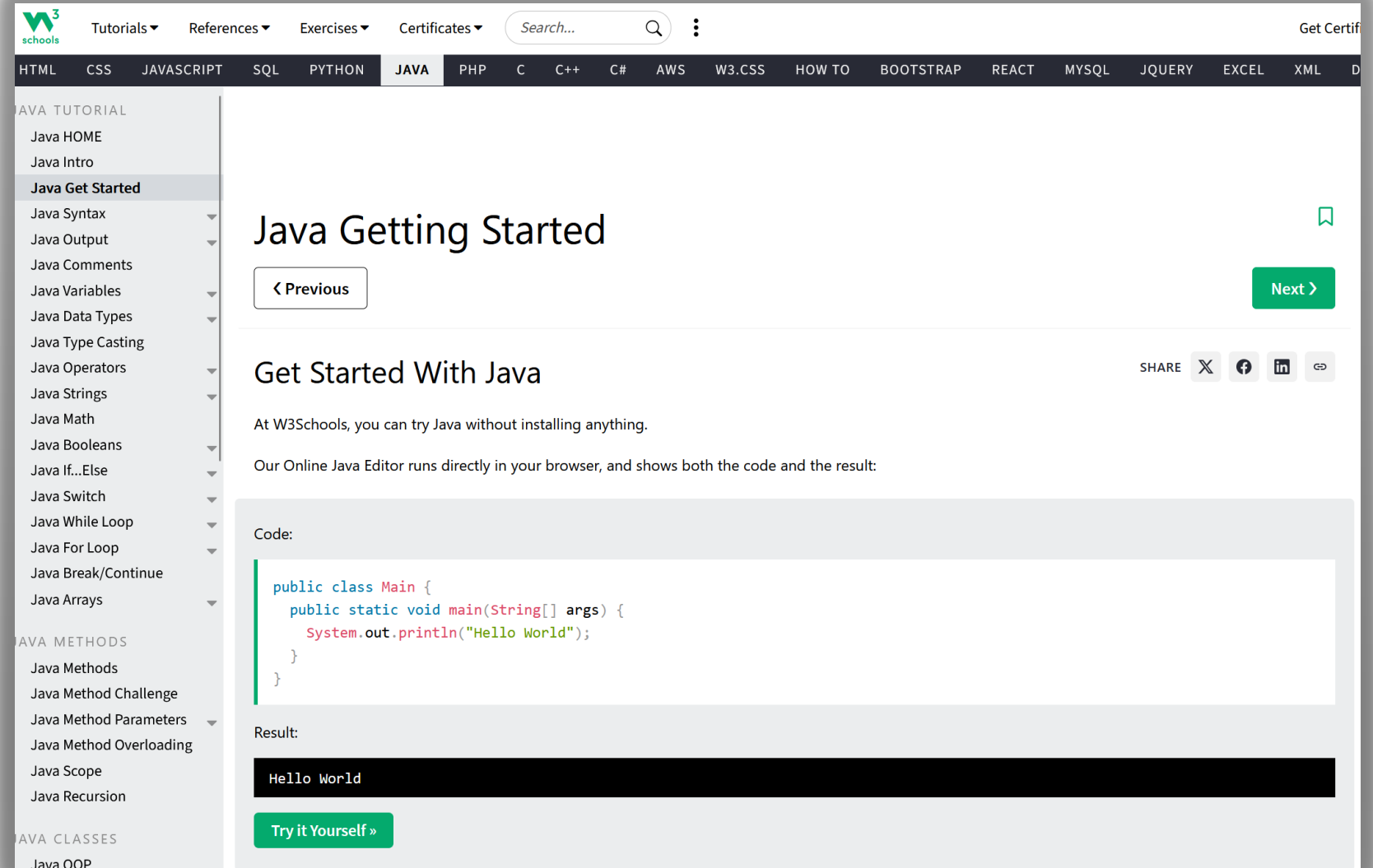
Kontrollstrukturen: Schleifen

Zusammenfassung

- Die **for-Schleife** wird meist verwendet, wenn die Anzahl der Durchläufe bekannt ist.
- Die **while-Schleife** prüft die Bedingung vor dem ersten Durchlauf und kann daher auch kein einziges Mal ausgeführt werden.
- Die **do while-Schleife** führt den Anweisungsblock immer mindestens einmal aus.
- Jede Schleife benötigt eine Bedingung oder eine Veränderung, damit sie nicht unbeabsichtigt endlos läuft.

die Programmiersprache Java

W3Schools



The screenshot shows the W3Schools website interface. At the top, there's a navigation bar with links for Tutorials, References, Exercises, and Certificates, along with a search bar. Below this is a dark blue header with various technology categories like HTML, CSS, JAVASCRIPT, SQL, PYTHON, JAVA (highlighted), PHP, C, C++, C#, AWS, W3.CSS, HOW TO, BOOTSTRAP, REACT, MYSQL, JQUERY, EXCEL, XML, and D. On the left side, a sidebar lists the Java tutorial topics, with 'Java Get Started' currently selected. The main content area is titled 'Java Getting Started' and includes a 'Previous' button and a 'Next' button. Below the title, it says 'Get Started With Java' and provides a brief introduction to Java, mentioning that it can be tried without installation and that an online editor is available. A code block shows a simple Java program that prints 'Hello World'. Below the code, the result 'Hello World' is displayed. At the bottom of the code block, there is a 'Try it Yourself' button.

W3schools Tutorials References Exercises Certificates Search... Get Certif

HTML CSS JAVASCRIPT SQL PYTHON **JAVA** PHP C C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUERY EXCEL XML D

JAVA TUTORIAL

- Java HOME
- Java Intro
- Java Get Started**
- Java Syntax
- Java Output
- Java Comments
- Java Variables
- Java Data Types
- Java Type Casting
- Java Operators
- Java Strings
- Java Math
- Java Booleans
- Java If...Else
- Java Switch
- Java While Loop
- Java For Loop
- Java Break/Continue
- Java Arrays

JAVA METHODS

- Java Methods
- Java Method Challenge
- Java Method Parameters
- Java Method Overloading
- Java Scope
- Java Recursion

JAVA CLASSES

- Java OOP

Java Getting Started

< Previous

Next >

Get Started With Java

SHARE X f in

At W3Schools, you can try Java without installing anything.

Our Online Java Editor runs directly in your browser, and shows both the code and the result:

Code:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Result:

Hello World

Try it Yourself »

<https://www.w3schools.com/java/default.asp>

die Programmiersprache Java

W3Schools

The screenshot displays the W3Schools website's Java tutorial page. The top navigation bar includes links for Tutorials, References, Exercises, and Certificates, along with a search bar. The left sidebar lists various Java topics, with 'Java Get Started' currently selected. The main content area features the title 'Java Getting Started' and a 'Get Started With Java' section. This section contains a brief introduction and a code editor with a 'Hello World' program. Below the code editor, the output 'Hello World' is shown. A green 'Try it Yourself »' button is prominently displayed at the bottom of the code editor area, highlighted with a red rectangular border.

W3schools Tutorials References Exercises Certificates Search... Get Certif

HTML CSS JAVASCRIPT SQL PYTHON **JAVA** PHP C C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUERY EXCEL XML D

JAVA TUTORIAL

- Java HOME
- Java Intro
- Java Get Started**
- Java Syntax
- Java Output
- Java Comments
- Java Variables
- Java Data Types
- Java Type Casting
- Java Operators
- Java Strings
- Java Math
- Java Booleans
- Java If...Else
- Java Switch
- Java While Loop
- Java For Loop
- Java Break/Continue
- Java Arrays

JAVA METHODS

- Java Methods
- Java Method Challenge
- Java Method Parameters
- Java Method Overloading
- Java Scope
- Java Recursion

JAVA CLASSES

- Java OOP

Java Getting Started

< Previous

Next >

Get Started With Java

SHARE X f in

At W3Schools, you can try Java without installing anything.

Our Online Java Editor runs directly in your browser, and shows both the code and the result:

Code:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Result:

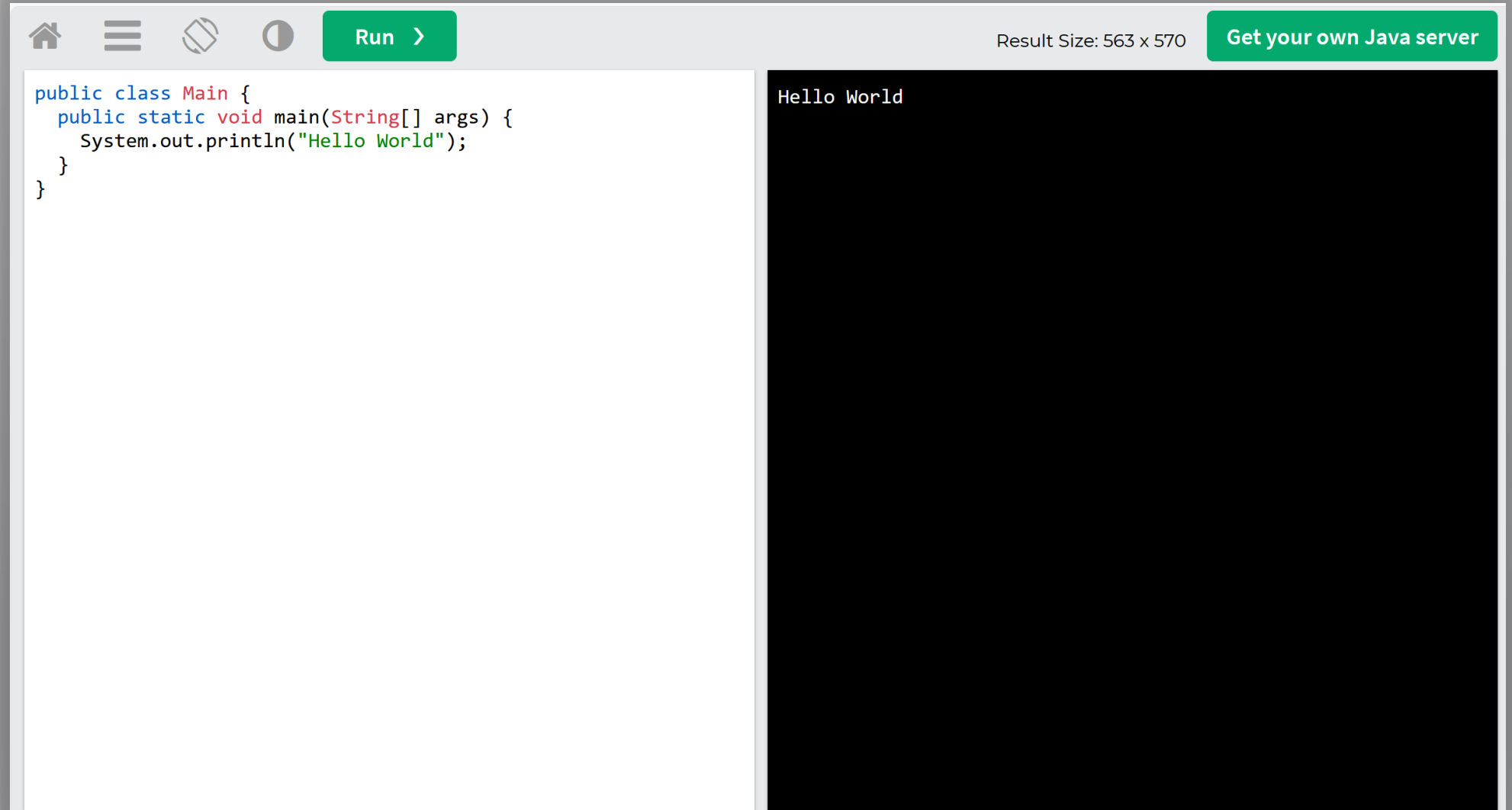
Hello World

Try it Yourself »

<https://www.w3schools.com/java/default.asp>

die Programmiersprache Java

W3Schools



The screenshot displays the W3Schools Java online compiler interface. At the top, there is a navigation bar with icons for home, menu, clipboard, and a toggle switch, followed by a green 'Run >' button. On the right side of the top bar, it says 'Result Size: 563 x 570' and a green button that says 'Get your own Java server'. The main area is split into two panels. The left panel contains the following Java code:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

The right panel shows the output of the program, which is 'Hello World'.

<https://www.w3schools.com/java/default.asp>